Project Design Document: Runtime Terror

Project Type: Infinite Runner / Puzzle Platformer

Tech Stack: C++ (Core Logic), Raylib/SFML (Rendering, subject to change)

Course Focus: Data Structures & Algorithms (DSA)

---

1. Executive Summary

"Runtime Terror" is an infinite side-scrolling platformer that gamifies the execution flow of a computer program. The player controls the **Instruction Pointer**, racing through an infinite script of C++ code. The goal is to execute code (run on platforms) while avoiding syntax errors, runtime exceptions, and memory leaks. The game features a unique **"Ctrl+Z" (Rewind)** mechanic powered by a custom data structure, allowing the player to undo fatal mistakes within a limited memory capacity.

---

2. The Concept & Logic

- **The Setting:** The game takes place inside a Code Editor (IDE) in "Dark Mode."
- **The Protagonist:** The player is the **Instruction Pointer** (a glowing block cursor or an arrow `->`).
- **The Objective:** Keep the program running. The "Codebase" is infinite. The further you travel, the more "Lines of Code" (Score) you execute.
- **The Failure State (Game Over):**
    1. **Crash:** Touching a "Syntax Error" (Red highlighted code).
    2. **SegFault:** Falling into a gap (Null Pointer).
    3. **Timeout:** Getting stuck behind an obstacle and falling off the left side of the screen (System Hang).

---

3. Gameplay Mechanics

A. Core Controls

- **Auto-Scroll:** The camera moves right at a constant speed. This represents the **Clock Speed** of the CPU. As the game progresses, the Clock Speed increases (Overclocking), forcing quicker reactions.
- **Movement:** Jump (Space) and Double Jump to navigate complex code structures (nested loops/if-statements).

B. The "Ctrl+Z" Rewind System

- **Logic:** Real-world editors have an "Undo" history. Our Instruction Pointer has a "Cache" that records previous states.
- **Mechanic:** If the player miscalculates a jump toward a fatal error, they can hold the **[Z]** key. The game pauses execution and reverses the player's position and velocity.
- **Limitation:** This is governed by a **"RAM Bar"**. Rewinding drains RAM. RAM is replenished by collecting "Garbage Collection" tokens (Green recycle icons).

---

4. World Design: The "Code Terrain"

The level is not made of rocks or grass; it is made of **Text**.

The Platforms (Safe Code)

Valid code lines are solid platforms. They are highlighted in standard syntax colors (Blue for keywords, White for variables).

- *Visual Example:* `if (isValid) {` or `while(true)`
- *Gameplay:* Standard solid ground.

The Hazards (The Errors)

Hazards are visually distinct code snippets representing programming errors.

Hazard Type	Visual Representation	Effect
Syntax Error	A block of code with a **jagged red underline** (squiggles).	**Instant Crash.** Touching this triggers a "Compilation Failed" Game Over.
Logic Error	A code block highlighted in **bright red background**.	**Damage/Slow.** Touching this doesn't kill you but slows you down, risking a "Timeout."
Null Gap	A literal gap in the text.	**SegFault.** Falling into the void ends the run.

The Exception	A floating red spiked ball (represented as throw e;).	**Dynamic Hazard.** Moves in a pattern (sine wave). Must be avoided.

---

5. Technical Architecture (DSA Implementation) (Suggestive and not final)

This section details how the project fulfills the University Data Structure requirements.

1. Map Generation: std::deque<CodeChunk>

- **Problem:** The world is infinite; we cannot load it all at once.
- **Solution:** We treat the screen as a sliding window.
    - The map is managed by a **Double-Ended Queue (Deque)**.
    - As the camera moves right, we pop_front() (unload) the old lines of code that went off-screen.
    - We push_back() (load) new procedurally generated lines of code to the right.

2. Time Rewind: Circular Buffer (Ring Buffer)

- **Problem:** Storing infinite history for "Ctrl+Z" will crash the RAM.
- **Solution:** A fixed-size array acting as a Circular Buffer.
    - We allocate memory for exactly 300 frames (5 seconds of gameplay).
    - New player positions overwrite the oldest positions using modulo arithmetic: index = (index + 1) % Capacity.
    - *Complexity:* $O(1)$ insertion and retrieval.

3. Syntax Generation: Weighted Randomness

- **Problem:** Random code needs to look semi-realistic.
- **Solution:** We use arrays of pre-defined strings categorized by difficulty.
    - EasyArray: "int x = 0;", "float y;"
    - HardArray: "template <typename T>", "void* ptr = &ref;"
    - The generator picks from these arrays based on the current score (Difficulty Level).

4. High Score: Sorted Linked List

- **Solution:** We store the top 10 runs. When the game launches, it reads a file into a Linked List and uses a Sorting Algorithm (e.g., Insertion Sort) to display the local leaderboard.

---

6. Visual Style & UI

- **Aesthetic:** "VS Code Dark Theme" or "Sublime Text."
- **Font:** Monospaced (Consolas, JetBrains Mono, or Courier).
- **UI Elements:**
    - **Score:** "Line: 402" (Top Left).
    - **Health/Mana:** "RAM Usage" (A bar at the top right).
    - **Game Over Screen:** A Pop-up window styled like a Windows Error Message or a Terminal Crash Log.